

UmojaFlowOS VASP Feature Assessment and Keycloak Rotation Architecture

Prepared by: Manus AI

Repository revision: fb1acc62e701ef3c74a52180802b3667646dc958

Assessment posture: Conditional readiness; production launch remains NO-GO pending real staging evidence and independent approval

Scope: VASP control architecture, Keycloak/Vault secret rotation, canary recovery, token compliance monitoring, and high-frequency Vault version-cleanup risks

Executive summary

UmojaFlowOS has a substantial VASP-oriented control foundation. The codebase includes cloud-agnostic Keycloak and MinIO/S3-compatible evidence architecture, Vault-managed secrets, release-SHA binding, digest-verified evidence publication, WORM/Object Lock controls, payment webhook security, guarded TigerBeetle integration, reconciliation workflows, AML/CFT/CPF and Travel Rule interfaces, operational monitoring, chaos and recovery runbooks, and a four-role independent sign-off model.

The platform is **not yet entitled to claim live VASP readiness solely from local code validation**. The remaining distinction is between software control implementation and externally verifiable operating evidence. Real staging must exercise PostgreSQL, Keycloak, Vault, MinIO or another S3-compatible WORM store, TigerBeetle, provider integrations, monitoring, alert routing, backup/restore, and recovery paths. E-01 through E-09 evidence must be bound to the release SHA and independently verified before production approval.

The Keycloak rotation design is fail-closed after activation of a new secret. It writes a primary Vault version, tests the real token-to-evidence-gateway path, and, if the primary canary fails, creates a new compensating Keycloak secret and writes a new Vault version with `operation=compensating_rollback`. It does not delete or overwrite earlier Vault versions. This preserves audit history but also means that high-frequency rotations can accumulate versions unless a separate, carefully guarded cleanup policy is implemented.

1. VASP feature assessment

Capability	Implemented control surface	Integration status	Readiness interpretation
Identity and access	Keycloak OIDC issuer/audience validation, JWKS validation, role checks, release/run binding, Vault policy fixtures, secret rotation workflow	Local and contract-tested; staging credentials and endpoint trust still required	Strong internal control; external evidence pending
Payment execution	Guarded provider execution paths, HMAC verification, timestamp freshness, replay resistance, CIDR enforcement, webhook reconciliation	Provider-specific runtime configuration is environment-dependent	No live launch until provider staging tests pass
Ledger	Official TigerBeetle client path, confirmed transfer primitives, reconciliation logic, failover and partition test harnesses	Real cluster ID, accounts, replicas, and network failure tests are required	Conditional until E-04 is complete
AML/CFT/CPF	Fail-closed screening contracts, timeout handling, case/evidence paths, monitoring hooks	External screening provider and controlled live test are not available in local validation	Conditional until E-05 evidence is complete
Travel Rule and regulatory reporting	Interfaces and evidence register structures exist	Real counterparty exchange and regulatory recipient tests remain external	Conditional; no regulator determination is inferred
Evidence and audit	Release mapping, SHA-256 body checks, signed manifests, WORM/Object Lock, detached signatures, audit records	Real immutable storage retention, tamper, and restore evidence required	Strong architecture; E-01/E-05/E-06/E-09 pending
Operations	Prometheus rules, Grafana dashboards, Alertmanager/PagerDuty integration patterns, circuit breakers, chaos runbooks	Live scrape, alert delivery, and recovery evidence required	Conditional until E-07/E-08 pass
Governance	Four independent roles: Release Manager, Security, Compliance, Operations; distinct payloads and segregation-of-duties controls	Verified enterprise subjects and signed approvals are required	Governance model present; approvals pending

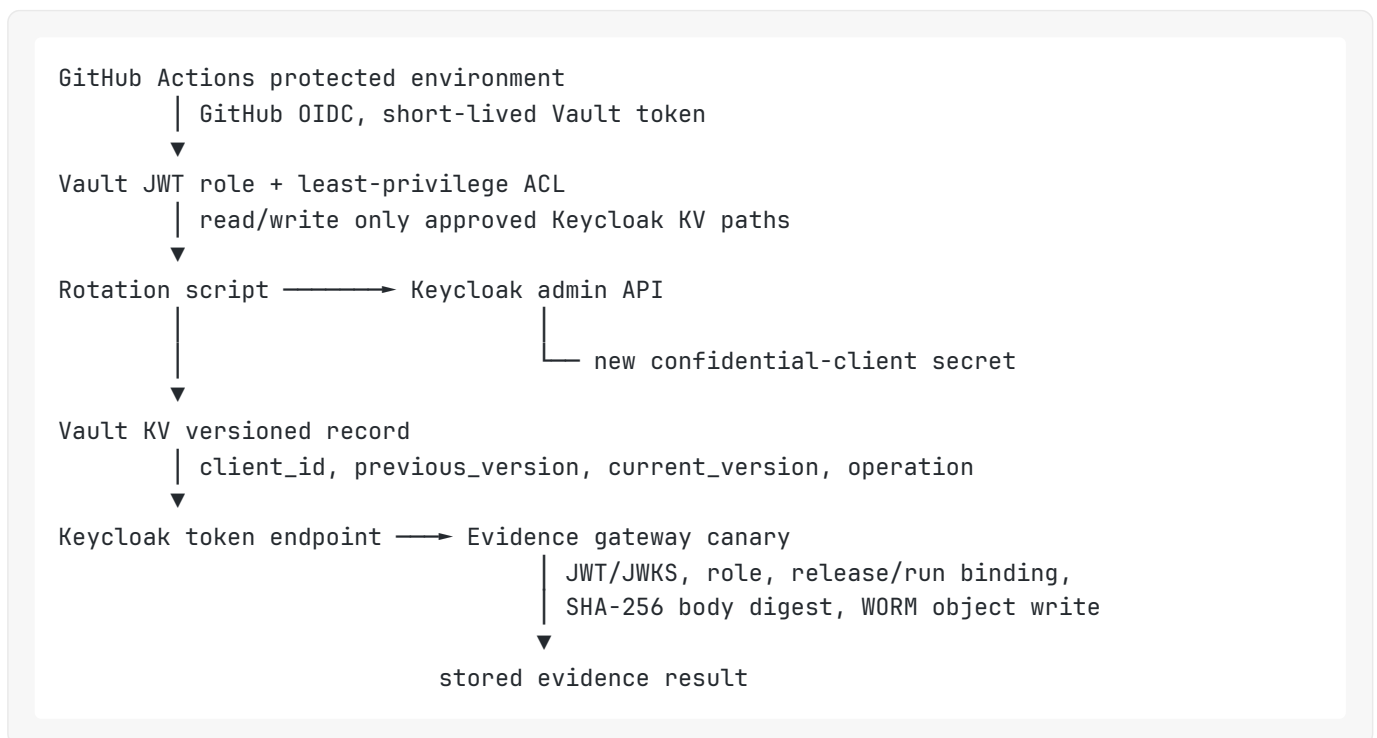
E-01 through E-09 evidence boundary

The authoritative staging commandbook identifies the following evidence groups:

Evidence	Required proof
E-01	Signed release tag, protected review, build provenance, SBOM, immutable image digests bound to <code>RELEASE_SHA</code>
E-02	PostgreSQL migration execution, schema and grants validation, and reconciliation-column checks
E-03	Real PostgreSQL application-role workflow integration and segregation-of-duties behavior
E-04	TigerBeetle transfer, reconciliation, partition, replica, and failover evidence
E-05	Keycloak, AML/CFT, webhook, regulatory, WORM, and notification integration evidence
E-06	Immutable deployment, health gates, rollback, and release-promotion evidence
E-07	Prometheus, Alertmanager, PagerDuty, and Grafana alerting evidence
E-08	Backup/restore, circuit-breaker, chaos, and recovery evidence
E-09	Security audit, independent review, and final sign-off evidence

2. Keycloak and Vault architecture

The architecture separates duties across the identity provider, secret manager, evidence gateway, and deployment workflow.



The evidence gateway validates the OIDC issuer, audience, RS256 algorithm, JWKS key ID, required role, and optional release/run claims. It also verifies that the request release SHA is active in a server-side mapping and that the evidence body digest matches `X-Evidence-SHA256` before writing to the immutable store.

The production client restrictions require a confidential client, service accounts only, short-lived access tokens, disabled direct-access grants, disabled browser flows, no offline access, and only the minimum evidence-publishing

role. Administrative Keycloak roles are excluded from the evidence-publisher client.

3. Primary rotation and rollback behavior

The Bash script starts with `set -euo pipefail` and `umask 077`. Required environment variables are checked before any external operation. It obtains an admin token, resolves exactly one Keycloak client, requests a new secret, masks it in CI logs, and sets `rotation_active=1` before the primary Vault write.

The primary Vault write records:

```
{
  "data": {
    "client_id": "<evidence-client>",
    "client_secret": "<new-secret>",
    "previous_version": "<prior-version>",
    "current_version": "<new-version>",
    "operation": "primary_rotation"
  }
}
```

The primary canary requests a token with the new secret and performs an authenticated PUT to the evidence gateway. The body includes the configured release SHA and run ID. The script computes a SHA-256 digest and requires the gateway response to return `status=stored` and the same digest.

If a command fails after `rotation_active=1`, the `ERR` trap invokes the compensating path:

```
primary operation fails
    ↓
rollback() is invoked
    ↓
generate another Keycloak secret
    ↓
write a new Vault version with compensating_rollback
    ↓
run the gateway canary using the recovery secret
    ├── pass: record ROTATION_ROLLED_BACK and stop
    └── fail: emit ROLLBACK_FAILED and exit nonzero
```

The recovery path is not a destructive rollback to an older Vault version. It is a forward compensating rotation. This is safer for auditability and avoids destroying evidence, but it requires consumers to obtain the current active version consistently and requires a separate lifecycle process for old-version cleanup.

4. Token compliance monitoring

`monitor_keycloak_token_compliance.py` performs client-credentials token acquisition, validates `expires_in` against configured minimum and maximum TTL, introspects the token, requires `active=true`, verifies issuer and audience, and optionally revokes a dedicated canary token before introspecting it again and requiring `active=false`.

A successful exposition includes:

```
umoja_keycloak_token_monitor_up{realm="...",client_id="..."} 1
umoja_keycloak_token_ttl_seconds{realm="...",client_id="..."} <seconds>
umoja_keycloak_token_revocation_check{realm="...",client_id="..."} 1
umoja_keycloak_token_monitor_failures_total{realm="...",client_id="..."} 0
```

A failed check returns exit code `1` and writes `monitor_up=0`, a failure metric, and duration. The scheduler must use an atomic temporary-file rename and remove stale metric files after an unsuccessful run. The revocation canary must use a separate least-privileged client because it intentionally invalidates its token.

5. High-frequency Vault version-cleanup failure analysis

The current rotation script deliberately does not delete old versions. That is the correct default during recovery, but it transfers cleanup responsibility to a separate lifecycle process. If rotations occur frequently, especially when every failed canary triggers an additional compensating rotation, the number of versions can grow faster than the nominal rotation frequency.

Failure-mode register

ID	Failure mode	Impact	Detection	Required mitigation
V-01	Version accumulation exceeds KV metadata or storage budget	Secret reads, backups, or Vault performance degrade; rotation can fail	Alert on version count, Vault storage growth, and write latency	Set a documented version-retention ceiling; alert before the ceiling; test cleanup in staging
V-02	Cleanup races an active rotation	A cleanup job may destroy the version currently needed by the gateway or rollback path	Correlate cleanup request with rotation lock/version metadata	Use a distributed rotation lease and mark active/current/rollback-protected versions before deletion
V-03	Cleanup races delayed consumers	A gateway pod, job, or recovery worker may still hold a previously fetched secret	Audit consumer leases and secret-fetch timestamps	Retain versions for at least the maximum token lifetime plus deployment propagation and recovery margin
V-04	Deleting the previous version before canary completion	Recovery loses a known-good audit reference and may become impossible	Compare cleanup timestamp with canary completion timestamp	Cleanup must exclude the current, previous, and any incident-protected versions until final verification
V-05	Compensating rotation creates repeated versions during an outage	An outage can create a primary/recovery pair on every retry, multiplying storage and operational ambiguity	Track primary failures and rollback failures separately	Apply bounded retry with exponential backoff, a circuit breaker, and a manual escalation threshold
V-06	KV metadata and secret payload become inconsistent	Operators may believe a version is current while consumers read a different version	Compare <code>current_version</code> , Vault metadata, and Keycloak client state	Use a transaction-like state record, monotonic version IDs, and post-write read-back verification
V-07	Cleanup removes	Security and compliance	Audit log retention and manifest references show missing versions	Apply legal/audit holds and prohibit cleanup

ID	Failure mode	Impact	Detection	Required mitigation
	forensic evidence too early	cannot reconstruct the rotation incident		of versions referenced by evidence or incidents
V-08	Cleanup job uses broad Vault permissions	A compromised or misconfigured job can delete unrelated secrets	Vault audit logs and policy review	Dedicated cleanup policy limited to one exact path and metadata operation; deny wildcard paths
V-09	Cleanup is not idempotent	Retries produce errors, partial deletion, or inconsistent state	Repeated-run tests and cleanup result metrics	Use a deterministic candidate list, version preconditions, and safe handling of already-removed candidates
V-10	Clock skew invalidates age-based retention	Versions are deleted too early or retained indefinitely	Compare node/Vault time and cleanup timestamps	Use Vault server timestamps where possible; enforce NTP and conservative retention buffers
V-11	Metrics remain stale after monitor failure	Prometheus sees an old success file and misses an outage	Compare metric timestamp/age with scheduler heartbeat	Atomic replacement plus stale-file deletion and a separate heartbeat metric
V-12	CI cleanup is mistaken for production cleanup	Ephemeral workflow cleanup does not protect or manage the production Vault store	Compare cleanup execution environment and Vault audit source	Production cleanup must run as an authenticated, reviewed service in the production monitoring namespace
V-13	Rollback version is cleaned before recovery review	A successful recovery canary is preserved, but the incident evidence is lost	Correlate <code>operation=compensating_rollback</code> with sign-off state	Retain every rollback version until incident closure and independent Security/Operations review

ID	Failure mode	Impact	Detection	Required mitigation
V-14	Secret values leak through process inspection or debug output	Credential compromise despite correct Vault versioning	Host audit, CI log scan, and shell tracing policy	Disable shell tracing, avoid command-line secrets, use protected files or memory injection, and mask CI values
V-15	Rotation succeeds in Vault but fails in Keycloak consumers	New secret is stored but application pods continue using an old value	Canary, gateway health, and secret-version labels	Require post-rotation consumer refresh/read-back and a bounded rollout completion gate
V-16	Cleanup deletes a version referenced by a signed release manifest	Evidence verification fails and audit chain is broken	Manifest-to-Vault version reference check	Treat signed manifest references as immutable legal holds until retention expiry and approval

6. Recommended cleanup protocol

A safe cleanup service should not be embedded into the emergency rotation script. It should be a separate, least-privileged, independently reviewed process with the following sequence:

1. Acquire a single-writer cleanup lease for the exact Vault path.
2. Read Vault metadata and the current rotation state.
3. Load active version, previous version, recovery version, incident holds, signed-manifest references, and consumer leases.
4. Calculate candidates using Vault server timestamps and a retention interval greater than token TTL, propagation time, and recovery margin.
5. Exclude current, previous, recovery, held, referenced, and recently accessed versions.
6. Re-read metadata immediately before each deletion and refuse to delete if the metadata changed.
7. Delete only the explicitly enumerated versions using an exact-path policy.
8. Re-read metadata and write an immutable cleanup audit record containing candidate versions, deleted versions, exclusions, policy version, and correlation ID.
9. Emit metrics for candidate count, deleted count, skipped count, conflicts, failures, and oldest retained version.
10. Freeze cleanup automatically when a rotation, canary, rollback, incident, or evidence-verification operation is active.

The cleanup service should be tested with concurrent primary rotation, failed primary canary, successful compensating recovery, repeated retries, delayed consumers, Vault unavailability, clock skew, and audit/legal holds. A cleanup failure must never block emergency recovery by deleting a secret; it should instead fail closed and alert.

7. Production decision

The platform is suitable for a controlled staging programme, not an unconditional live VASP launch. The correct decision is:

GO for controlled staging and independent evidence collection; NO-GO for live customer payment and regulatory activation until E-01 through E-09 and the four independent approvals are complete.

The most important remaining assurance item for the Keycloak/Vault path is not the existence of the rotation script. It is proof that the real staging environment can execute primary rotation, deliberately fail the primary canary, complete compensating recovery, preserve Vault history, emit durable metrics, and recover without allowing stale or unauthorized credentials to publish evidence.

Repository evidence

Evidence	Repository path
Rotation and rollback implementation	<code>scripts/infra/rotate_keycloak_evidence_secret.sh</code>
Token TTL/revocation monitor	<code>scripts/infra/monitor_keycloak_token_compliance.py</code>
Rotation policy	<code>infra/vault/policies/umoja-keycloak-evidence-rotation.hcl</code>
Keycloak production client restrictions	<code>infra/evidence-gateway/keycloak/production-client-restrictions.json</code>
Prometheus alerts	<code>infra/monitoring/keycloak-token-compliance-alerts.yml</code>
Grafana dashboard	<code>infra/monitoring/grafana/keycloak-token-compliance-dashboard.json</code>
Staging workflow	<code>.github/workflows/staging-keycloak-rotation-validation.yml</code>
Staging commandbook	<code>assurance/latest_commit_staging_evidence_commandbook.md</code>
Rotation and monitoring runbook	<code>assurance/keycloak_rotation_and_token_monitoring_runbook.md</code>
Defect-discovery ground truth	<code>assurance/defect_discovery/ground_truth_maps.md</code>

Note: This report is an internal engineering readiness assessment. It is not a legal opinion, regulator determination, or certification of VASP authorization.